

HTML, CSS

Internet Basics

Topics To Be Covered



PATH to HTML

Basic Structure

Semantic
Elements

Multimedia
Elements

Form
Elements

Canvas and
SVG

Geolocation
API

Local Storage
and Session
Storage

Responsive
Design

Accessibility

Cross-Browser
Compatibility

Validate Your
HTML

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Document Type Declaration (DOCTYPE):

1.The `<!DOCTYPE>` declaration is used at the very beginning of an HTML document to inform the web browser which version of HTML is being used. For HTML5, the doctype declaration is simply `<!DOCTYPE html>`.

HTML Element:

1.The `<html>` element serves as the root element of the HTML document and encapsulates all other elements. It typically includes two main sections: `<head>` and `<body>`.

Head Section (`<head>`):

1.The `<head>` section contains metadata and other information about the HTML document, such as the document's title, character encoding, viewport settings, links to external resources (like stylesheets and scripts), and other meta tags for SEO (Search Engine Optimization) purposes.

Document Title (`<title>`):

1.The `<title>` element, located within the `<head>` section, specifies the title of the web page. The title appears in the browser's title bar or tab and is also used by search engines for indexing purposes.

Path to HTML



Basic Structure		
Semantic Elements		
Multimedia Elements		
Form Elements		
Canvas and SVG		
Geolocation API		
Local Storage and Session Storage		
Responsive Design		
Accessibility		
Cross-Browser Compatibility		
Validate Your HTML		
	Viewport Configuration (<meta>):	1.The <meta> elements within the <head> section are used to specify metadata about the HTML document, such as the character encoding (charset), viewport settings for responsive design (viewport), author information, and other custom metadata.
	External Resources (CSS, JavaScript, etc.):	1.Links to external resources, such as CSS (Cascading Style Sheets) files for styling and JavaScript files for interactivity, are often included within the <head> section using the <link> and <script> elements, respectively.
	Body Section (<body>):	1.The <body> section contains the main content of the web page, including text, images, multimedia elements, forms, and other HTML elements that make up the visual and interactive components of the page.
	HTML Comments (<!-- -->):	1.HTML comments are used to add explanatory notes or annotations within the HTML code without affecting the rendered output in the browser. Comments are enclosed within <!-- and --> delimiters.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Semantic elements in HTML5 are elements that provide meaning to the content they enclose, making the structure of the document more understandable for both humans and machines. These elements describe the purpose or role of the content they contain, rather than just its appearance.

Using semantic elements properly improves accessibility, search engine optimization (SEO), and the overall maintainability of the code.

Semantic Elements

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Semantic Elements

<header>: The `<header>` element typically contains introductory content or navigational links for its parent section or document. It often includes things like logos, headings, and navigation menus.

<nav>: The `<nav>` element represents a section of a page that contains navigation links, such as menus, tables of contents, or indexes.

<main>: The `<main>` element represents the main content of the document or a section within it. It should not include content that is repeated across multiple pages, such as navigation links or footers.

<section>: The `<section>` element represents a thematic grouping of content within a document. It can be used to divide the content into different sections, such as chapters, headers, footers, or any other groups of related content.

<article>: The `<article>` element represents a self-contained piece of content that can be distributed and reused independently. It's often used for blog posts, news articles, forum posts, or comments.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Semantic Elements

<aside>: The `<aside>` element represents content that is tangentially related to the main content of the page, such as sidebars, pull quotes, or advertisements.

<footer>: The `<footer>` element typically contains information about its containing element, such as copyright information, contact details, or links to related documents.

<figure> and **<figcaption>**: The `<figure>` element is used to encapsulate media content, such as images, videos, or illustrations, along with their captions, which are represented by the `<figcaption>` element.

<details> and **<summary>**: The `<details>` element is used to create a disclosure widget that can be toggled open and closed to reveal additional content. The summary of the additional content is provided by the `<summary>` element.

<time>: The `<time>` element represents a specific point in time or a range of time. It can be used to mark up dates, times, or durations in a way that is machine-readable and accessible to assistive technologies.

Path to HTML

Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

HTML provides several multimedia elements that allow you to embed various types of media content, such as images, audio, video, and more, directly into your web pages.

Multimedia Elements

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Multimedia Elements

****: The element is used to embed images into a web page. It's a self-closing tag and requires the attribute to specify the URL of the image file.

<audio>: The element is used to embed audio files, such as music or sound effects, into a web page. You can specify multiple source files using the element to ensure compatibility across different browsers.

<video>: The element is used to embed video files into a web page. Like the element, you can specify multiple source files using the element for cross-browser compatibility.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Multimedia Elements

<iframe>: The element is used to embed content from another web page or external source, such as a YouTube video or a Google Map.

<canvas>: The element is used to draw graphics, animations, or other visual content dynamically using JavaScript.

<source>: The element is used within and elements to specify alternative media resources. This is useful for providing different formats of the media file for compatibility with different browsers.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

HTML5 introduced several new form elements and attributes that provide enhanced functionality and improve the user experience when creating web forms.

Form Elements

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Form Elements

<input>: The element is used to create various types of form controls, such as text fields, checkboxes, radio buttons, buttons, and more. The attribute determines the specific type of control.

<textarea>: The element is used to create a multiline text input field, allowing users to enter multiple lines of text.

<select> and **<option>**: The element creates a dropdown list, while the element defines individual options within the list.

<datalist> and **<option>**: The element provides a predefined list of options for an element with the attribute.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Form Elements

<fieldset> and **<legend>**:
The element groups related form controls together, while the element provides a caption or description for the .

<input type="email">,
<input type="url">,
<input type="tel">,
<input type="number">,
etc.: These input types provide specialized controls for specific types of input, such as email addresses, URLs, phone numbers, and numbers.

<input type="date">,
<input type="time">,
<input type="datetime-local">, etc.: These input types provide controls for selecting dates, times, and date-time combinations.

Path to HTML

Basic Structure
Semantic Elements
Multimedia Elements
Form Elements
Canvas and SVG
Geolocation API
Local Storage and Session Storage
Responsive Design
Accessibility
Cross-Browser Compatibility
Validate Your HTML

In HTML5, both and SVG (Scalable Vector Graphics) are used for creating graphics and visual content on web pages, but they serve different purposes and have distinct features.

Canvas and SVG

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Canvas

`<canvas>` is a HTML5 element that provides a drawable region on the web page using JavaScript. It allows you to draw graphics, animations, and other visual elements dynamically through JavaScript code. The `<canvas>` element doesn't have its own drawing capabilities; instead, you use JavaScript to draw shapes, paths, images, and text onto it.

Key features of `<canvas>`:

- It's resolution-dependent, meaning the drawing's quality depends on the resolution of the canvas.
- The drawing is rendered pixel by pixel, making it suitable for dynamic and interactive graphics.
- You can use methods like `getContext('2d')` to obtain a rendering context and then use methods like `fillRect()`, `strokeRect()`, `drawImage()`, `beginPath()`, `moveTo()`, `lineTo()`, etc., to draw and manipulate graphics.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

SVG (Scaler Vector Graphics)

SVG is an XML-based markup language for describing vector graphics, which can be scaled to any size without losing quality. It allows you to create graphics, shapes, and animations declaratively using XML syntax. Unlike `<canvas>`, SVG retains the vector nature of the graphics, making it ideal for creating resolution-independent and interactive graphics.

Key features of SVG:

- It's resolution-independent, allowing graphics to be scaled without loss of quality.
- SVG elements can be manipulated using CSS and JavaScript, making them highly customizable and interactive.
- SVG supports a wide range of graphic elements like `<rect>`, `<circle>`, `<ellipse>`, `<line>`, `<polyline>`, `<polygon>`, `<path>`, etc., which can be styled and animated using CSS or JavaScript.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

In HTML5, the Geolocation API provides a way to obtain the geographic location of a device, such as a smartphone, tablet, or computer, directly from a web browser. This enables web applications to offer location-based services, such as finding nearby businesses, displaying maps, or providing customized content based on the user's location.

Geolocation API

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Geolocation API

Getting User Consent

Before accessing the user's location, web applications must request permission from the user. Browsers typically prompt the user to grant or deny permission to share their location.

Accessing the Geolocation API

Once the user grants permission, JavaScript code within the web page can access the Geolocation API provided by the browser.

Retrieving the Location

The Geolocation API provides methods to retrieve the user's current location. The most common method is navigator.

`\geolocation.`

`\getCurrentPosition()`, which retrieves the current position of the device.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Geolocation API

Handling the Location Data

Once the location is retrieved successfully, the function is called with a object. This object contains properties such as `latitude` and `longitude`, which represent the latitude and longitude coordinates of the user's location.

Error Handling

If an error occurs during location retrieval (e.g., the user denies permission or the browser is unable to determine the location), the function is called with a object containing information about the error.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Geolocation API

Handling Privacy Concerns

It's important to handle user privacy concerns when using the Geolocation API. Always inform users about why their location is being requested and how it will be used. Respect user preferences and provide options to disable or limit location tracking.

Using the Location Data

Once the location data is obtained, web applications can use it to provide location-based services, such as displaying maps, finding nearby points of interest, or customizing content based on the user's location.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

In HTML5, the Web Storage API provides two mechanisms for storing data locally in the user's web browser: Local Storage and Session Storage. Both of these mechanisms allow web applications to store data persistently on the client side, enabling them to retain state information even after the browser is closed or the page is refreshed.

Location and Session Storage

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Location Storage

Purpose: Local Storage is designed for storing data with no expiration date. The data stored in Local Storage remains available even after the browser is closed and reopened.

Storage Limit: Local Storage typically provides more storage space (usually up to 5MB) compared to Session Storage.

Usage: To store data in Local Storage, you can use the localStorage object, which provides methods like `setItem()`, `getItem()`, `removeItem()`, and `clear()`.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Session Storage

Purpose: Session Storage is designed for storing data with a limited lifetime. The data stored in Session Storage is available only for the duration of the page session. Once the browser tab or window is closed, the data is cleared.

Storage Limit: Session Storage typically provides a smaller storage space compared to Local Storage, and the exact limit varies between browsers.

Usage: To store data in Session Storage, you can use the sessionStorage object, which provides methods similar to those of Local Storage

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Some words

Both Local Storage and Session Storage provide a simple key-value storage mechanism and are accessible through the `localStorage` and `sessionStorage` objects, respectively. They can be used to store various types of data, including strings, numbers, arrays, and objects.

However, it's important to note that data stored in Local Storage and Session Storage is accessible to any JavaScript code running in the same origin (domain, protocol, and port), so sensitive data should be encrypted or handled with caution to prevent unauthorized access.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive web design is an approach to web development that ensures web pages render well on various devices and screen sizes. With the increasing use of smartphones, tablets, and other devices to access the internet, it's essential for websites to adapt to different viewport sizes and orientations.

Responsive Design

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

Mobile-First Approach:

Start with designing and developing for mobile devices first, and then progressively enhance the layout and features for larger screens using media queries. This approach ensures a better user experience on smaller devices.

Flexbox and CSS Grid:

Use CSS layout techniques like Flexbox and CSS Grid to create flexible and responsive layouts that adapt to different screen sizes and orientations. These layout methods provide powerful tools for building complex responsive designs.

Testing Across Devices:

Test your responsive designs across various devices and screen sizes to ensure they function correctly and look good on different platforms. Use browser developer tools, online emulators, and real devices for testing.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

Viewport Meta Tag:

Use the <meta> tag with the viewport attribute to control the layout and scaling of the web page on mobile devices. Setting the viewport meta tag ensures that the browser renders the page appropriately on different screen sizes.

Flexible Layouts with CSS:

Use relative units like percentages (%) or viewport-relative units like vw and vh for widths, heights, margins, and paddings. Avoid using fixed widths or heights to allow content to adjust based on the viewport size.

Media Queries:

Media queries allow you to apply different styles based on the characteristics of the device, such as screen width, height, orientation, and resolution. Use media queries to create responsive designs that adapt to various viewport sizes.

Responsive Images:

Use the max-width: 100%; height: auto; CSS rule for images to ensure they scale down proportionally on smaller screens. Additionally, consider using the srcset attribute to provide multiple image sources with different resolutions for better performance.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

html

 Copy code

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

CSS

 Copy code

```
.container {  
  width: 100%;  
  max-width: 1200px; /* Optional: Set maximum width */  
  margin: 0 auto; /* Center the container horizontally */  
}
```

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

CSS

 Copy code

```
@media screen and (max-width: 768px) {  
  /* Styles for screens up to 768px wide */  
}  
  
@media screen and (min-width: 768px) and (max-width: 1024px) {  
  /* Styles for screens between 768px and 1024px wide */  
}  
  
@media screen and (min-width: 1024px) {  
  /* Styles for screens wider than 1024px */  
}
```

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

CSS

 Copy code

```
@media screen and (max-width: 768px) {  
  /* Styles for screens up to 768px wide */  
}  
  
@media screen and (min-width: 768px) and (max-width: 1024px) {  
  /* Styles for screens between 768px and 1024px wide */  
}  
  
@media screen and (min-width: 1024px) {  
  /* Styles for screens wider than 1024px */  
}
```

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Responsive Design

html

Copy code

```

```

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Accessibility in HTML5 refers to the practice of making web content and applications usable and accessible to all users, including those with disabilities. HTML5 provides several features and techniques to improve accessibility and ensure that web content can be easily accessed, understood, and interacted with by everyone.

Accessibility

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Accessibility

Semantic HTML: Use semantic HTML elements to provide meaningful structure and context to your content. Elements like `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, and `<footer>` help screen readers and other assistive technologies understand the purpose and relationship of different parts of the page.

Proper Document Structure: Ensure that your HTML documents have a logical and hierarchical structure. Use headings (`<h1>`, `<h2>`, etc.) to organize content and provide a hierarchical outline of the page. Proper nesting of elements and meaningful use of labels and captions help users navigate and understand the content.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Accessibility

Text Alternatives for Images: Always include descriptive text alternatives (alt attributes) for images using the element. Alt text provides context and information about the content of the image for users who cannot see it, such as those using screen readers or with slow internet connections.

html

 Copy code

```

```

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Accessibility

Accessible Forms: Ensure that forms are accessible by using appropriate form controls, labels, and instructions. Use the `<label>` element to associate labels with form controls, provide helpful error messages, and ensure that form elements are keyboard accessible and usable with screen readers.

Keyboard Accessibility: Make sure that all interactive elements on your page, such as links, buttons, and form controls, are accessible via keyboard navigation. Users should be able to navigate through your website, interact with elements, and perform actions using only the keyboard.

Focus Management: Ensure that the focus state is clearly visible and consistent across all interactive elements on the page. Users should be able to easily identify which element has keyboard focus and navigate through the content without losing track of their position.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Accessibility

Color Contrast: Use sufficient color contrast between text and background colors to ensure readability for users with low vision or color vision deficiencies. The Web Content Accessibility Guidelines (WCAG) recommend a minimum contrast ratio of 4.5:1 for normal text and 3:1 for large text.

Semantic Markup for Links: Use descriptive and meaningful text for link anchor text. Avoid using generic phrases like "click here" or "read more" and instead provide contextually relevant link text that describes the destination or purpose of the link.

Accessible Media Players: If you include audio or video content on your website, provide accessible media players with controls for playback, volume, and captions. Ensure that transcripts and captions are available for users who cannot hear the audio content.

Testing with Accessibility Tools: Regularly test your website for accessibility using automated accessibility testing tools, browser developer tools, and screen readers. Conduct user testing with individuals with disabilities to identify and address accessibility barriers effectively.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Cross-browser compatibility in HTML5 refers to the ability of a website or web application to function consistently and correctly across different web browsers, such as Chrome, Firefox, Safari, Edge, and others. Given the variety of browsers available, each with its own rendering engine and implementation quirks, ensuring cross-browser compatibility is essential for reaching a wide audience and providing a consistent user experience.

Cross-Browser Compatibility

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Cross-Browser Compatibility

Use Modern HTML5 Features: Stick to standard HTML5 elements, attributes, and APIs supported by all modern browsers. Avoid using outdated or non-standard features that may not be supported in certain browsers.

Test Across Multiple Browsers: Test your website or web application in various browsers and their different versions to identify any compatibility issues. Use browser testing tools, online services, or virtual machines to simulate different browser environments.

Normalize CSS: Use a CSS normalization or reset stylesheet to ensure consistent rendering of HTML elements across browsers. Normalize.css and Reset.css are popular choices for resetting browser styles to a consistent baseline.

Prefix Vendor-Specific CSS Properties: When using CSS properties with experimental or vendor-specific prefixes (e.g., -webkit-, -moz-, -ms-, -o-), ensure that you include the unprefixed version as well. This helps ensure compatibility with browsers that support the standard version of the property.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Cross-Browser Compatibility

Use Feature Detection:

Implement feature detection techniques, such as feature queries (@supports), Modernizr, or JavaScript feature detection, to detect browser capabilities and provide fallbacks or polyfills for unsupported features.

Progressive Enhancement:

Follow the principle of progressive enhancement by starting with a baseline of functionality that works across all browsers and then adding enhancements for modern browsers that support additional features. This ensures that the core functionality remains accessible to all users.

Test Responsiveness:

Ensure that your website or web application is responsive and adapts to different screen sizes and devices. Test responsiveness across various devices, including desktops, laptops, tablets, and smartphones, to ensure a consistent user experience.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Cross-Browser Compatibility

Handle JavaScript Cross-Browser Issues: Be aware of JavaScript compatibility issues and browser quirks when writing client-side JavaScript code. Use feature detection, polyfills, and libraries like jQuery to mitigate cross-browser JavaScript compatibility issues.

Regularly Update and Maintain: Stay updated with the latest browser versions and standards, and regularly update your codebase to address any compatibility issues or deprecated features. Keep an eye on browser vendor announcements and developer documentation for any changes or updates.

User-Agent Detection (As a Last Resort): Use user-agent detection as a last resort to apply specific browser-specific fixes or workarounds. However, this approach should be used sparingly, as user-agent strings can be spoofed and may not accurately reflect a browser's capabilities.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Validating HTML is the process of checking your HTML code against the official specifications set by the W3C (World Wide Web Consortium) to ensure that it follows the correct syntax and structure. Valid HTML is essential for creating web pages that display correctly across different browsers and devices.

Validate Your HTML

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Validate Your HTML

W3C Markup Validation Service: The W3C provides an online tool called the Markup Validation Service, which allows you to validate HTML documents by entering the URL or directly pasting the HTML code. It checks for syntax errors, deprecated elements, and other issues that may affect the rendering of your web page.

Using Browser Developer Tools: Most modern web browsers come with built-in developer tools that include HTML validation features. You can use these tools to inspect the HTML code of your web page and identify any syntax errors or issues. For example, in Google Chrome, you can access the "Elements" panel of the developer tools to view and validate the HTML code.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Validate Your HTML

Integrated Development Environments (IDEs): Many code editors and IDEs (Integrated Development Environments) include HTML validation features as part of their toolset. These tools often provide real-time feedback and suggestions to help you write valid HTML code as you work. Examples include Visual Studio Code, Sublime Text, and Atom.

Browser Extensions: There are browser extensions available that can validate HTML directly within the browser. These extensions typically add a button or menu option to the browser toolbar, allowing you to validate the current web page with a single click.

Path to HTML



Basic Structure

Semantic Elements

Multimedia Elements

Form Elements

Canvas and SVG

Geolocation API

Local Storage and Session Storage

Responsive Design

Accessibility

Cross-Browser Compatibility

Validate Your HTML

Validate Your HTML

Online Validators and Linters:

Besides the W3C Markup Validation Service, there are several other online validators and linters available that can check your HTML code for errors and best practices. Some popular options include HTMLHint, HTML Validator (WebExtensions), and Nu Html Checker.

Manual Inspection: You can also manually inspect your HTML code to look for common mistakes and issues, such as unclosed tags, missing attributes, and deprecated elements. Reviewing your code regularly and following best practices can help maintain its validity.

Topics To Be Covered

CSS PATH

- 1 Understand the Basics
- 2 Set Up Your Environment
- 3 Syntax and Selectors
- 4 Styling Properties
- 5 Box Model
- 6 Layout Techniques
- 7 Responsive Design

Path to CSS

Understand the Basics

Set Up Your Environment

Syntax and Selectors

Styling Properties

Box Model

Layout Techniques

Responsive Design

Q & A

Please ask your questions.